Reviewer Pack — Plethysm Coefficient Inverse Proportionality to SOS Refutati…

Entry 76c4f54f8fcf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 02:24:12 UTC

Plethysm Coefficient Inverse Proportionality to SOS Refutation Size for Symmetric CSPs

Entry ID: `76c4f54f8fcf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 02:24:12 UTC

1. Conjecture

Field A (mathematical branch): Plethysm Theory **Field B** (complexity object): SOS Refutation Size for CSPs

Statement:

> For a symmetric Boolean CSP instance Φ with n variables, the plethysm coefficient χ(Φ) of its characteristic polynomial satisfies χ(Φ) ≥ c · (log n) / (SOS_refutation_size(Φ)), where c > 0 is a constant. Equality holds when Φ is a symmetric constraint satisfaction problem with uniform clause distribution.

Rationale (proposer's reasoning):

> Plethysm coefficients capture the complexity of decomposing symmetric representations, while SOS refutation size measures the hierarchy's ability to handle symmetric constraints. Their inverse proportionality suggests structural barriers where symmetric algebraic invariants limit SOS efficiency.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0de45cf36fa71c58`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def plethysm_coefficient(n):
        if n == 0:
            return 1
        coeff = 0
        for k in range(1, n + 1):
            coeff += (-1) ** (n - k) * Fraction(math.comb(n, k)) * plethysm_coefficient(k)
        return coeff

    def sos_refutation_size(n):
        # Placeholder function to simulate SOS refutation size
        # This is a dummy implementation and should be replaced with actual computation
        return n ** 2

    n = random.randint(5, 40)
    chi = plethysm_coefficient(n)
    sos_size = sos_refutation_size(n)
    c = Fraction(1, 10)  # Placeholder constant, adjust as needed

    metric_value = chi * sos_size
    instances_tested = 1
    conjecture_holds = metric_value >= c * math.log(n)
    counterexample = "" if conjecture_holds else f"n={n}, chi*{sos_size} < {c*math.log(n)}"

    return {
        "metric_name": "plethysm_coefficient * sos_refutation_size",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18142075.py", line 58, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18142075.py", line 35, in run_trial
    chi = plethysm_coefficient(n)
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18142075.py", line 26, in plethysm_coef
    coeff += (-1) ** (n - k) * Fraction(math.comb(n, k)) * plethysm_coefficient(k)
                                                           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18142075.py", line 26, in plethysm_coef
    coeff += (-1) ** (n - k) * Fraction(math.comb(n, k)) * plethysm_coefficient(k)
                                                           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18142075.py", line 26, in plethysm_coef
    coeff += (-1) ** (n - k) * Fraction(math.comb(n, k)) * plethysm_coefficient(k)
                                                           ^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 993 more times]
  File "/usr/lib/python3.12/fractions.py", line 630, in reverse
    return monomorphic_operator(Fraction(a), b)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 754, in _mul
    return Fraction._from_coprime_ints(na * nb, db * da)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with recursion depth error, preventing evaluation of conjecture validity | next: Implement memoization for plethysm_coefficient recursion or increase recursion limit

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43175
2	preregistration	ollama_remote	qwen3:8b	0	0	24190
3	novelty	ollama_remote	qwen3:8b	0	0	20666
4	novelty	ollama_remote	qwen3:8b	0	0	13128
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19330
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7214
7	judge	ollama_remote	qwen3:8b	0	0	15798

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 143501 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/76c4f54f8fcf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/76c4f54f8fcf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/76c4f54f8fcf.tar.gz` (if generated)